BTS SIO — Option SISR

Travaux Pratiques Linux / Administration Système

Révision / LAB 1

ALEXANDRE MAQUANT SISR ST REMI

Commandes fondamentales Linux

Système · Fichiers · Répertoires · Liens · Navigation

Durée estimée	2h – 2h30
Objectif	Maîtriser les commandes de base du shell Linux
OS cible	Ubuntu 24.04 LTS

Pensez à prendre des captures d'écran et d'ajouter des explications afin de simplifier l'intégration sur votre portfolio.

Introduction au shell Linux

> Qu'est-ce que le shell ? Le shell est l'interface en ligne de commande (CLI) de Linux.
> Vous saisissez des commandes texte, le système les exécute et affiche le résultat.
> C'est l'outil fondamental de tout administrateur système Linux.

Anatomie d'une commande Linux

```
# Structure générale :
$ commande   [options]   [arguments]

# Exemple décomposé :
$ ls         -l          /home/student
#  ↑          ↑              ↑
#  commande  option        argument

# Le symbole $ indique l'invite de commande (prompt) utilisateur
# Ne pas le taper ! Il représente "le terminal attend votre commande"
```

Symbole	Signification
$	Invite de commande utilisateur standard (ne pas taper)
#	Invite root (super-administrateur) OU commentaire dans le code

Symbole	Signification
~	Raccourci pour votre répertoire personnel (/home/votre_login)
..	Répertoire parent (niveau au-dessus)
.	Répertoire courant

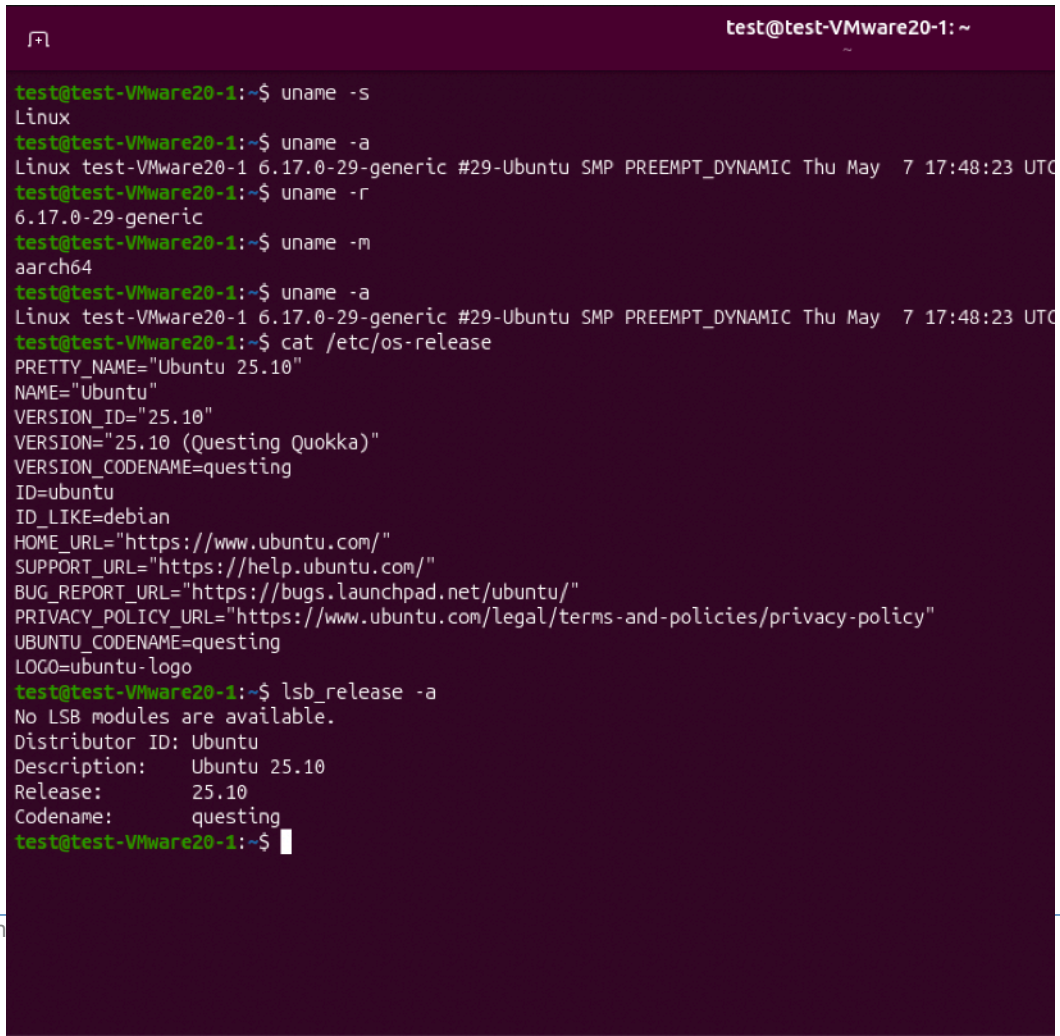
Astuce :
- Utilisez la touche Tab pour auto-compléter les noms de commandes et de fichiers.
- Utilisez les flèches ↑↓ pour naviguer dans l'historique des commandes.

Partie A — Informations système et date

1 Obtenir des informations sur le système

Cette tâche vous apprend à interroger votre système Linux :
- quel OS tourne,
- quel est votre identifiant,
- quel est le nom de la machine.

(i) Afficher le nom du système d'exploitation



```
# uname affiche des informations sur le noyau et le système
$ uname -s

# Sortie attendue :
Linux

# Pour plus de détails (version complète du noyau)
$ uname -a

# Nom de la distribution (Ubuntu, Debian...)
$ cat /etc/os-release
$ lsb_release -a
```

Option uname	Affiche
-s	Nom du noyau (Linux)
-r	Version du noyau
-m	Architecture processeur (x86_64, arm64...)
-a	Toutes les informations ci-dessus

(ii) Afficher le nom de connexion (login)

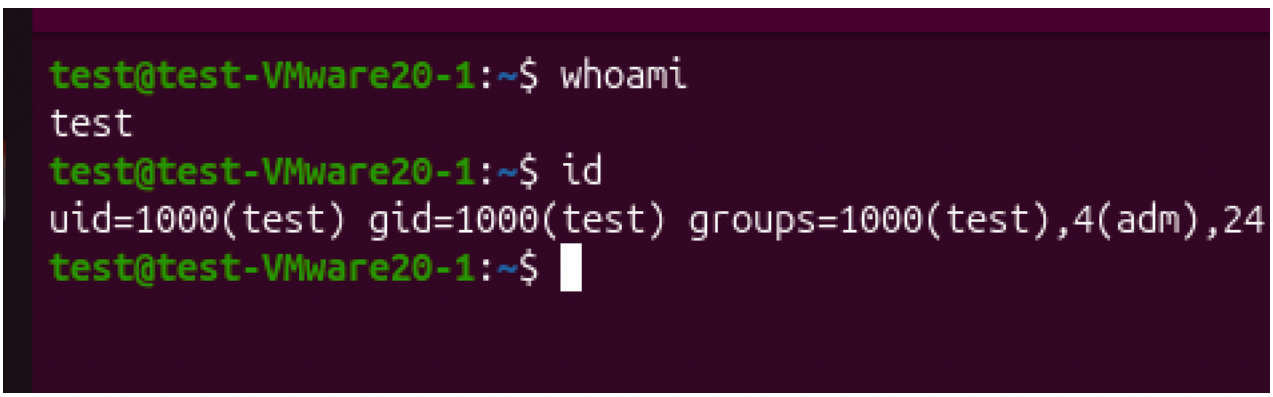
> whoami signifie littéralement "Qui suis-je ?"
> Il retourne le nom de l'utilisateur actuellement connecté dans le terminal.

```
# Nom de l'utilisateur courant
$ whoami

# Sortie attendue :
etudinfo

# Informations complètes sur l'utilisateur (UID, groupes...)
$ id

# Sortie attendue :
uid=1000(etudinfo) gid=1000(etudiant) groups=1000(etudiant),27(sudo)
```



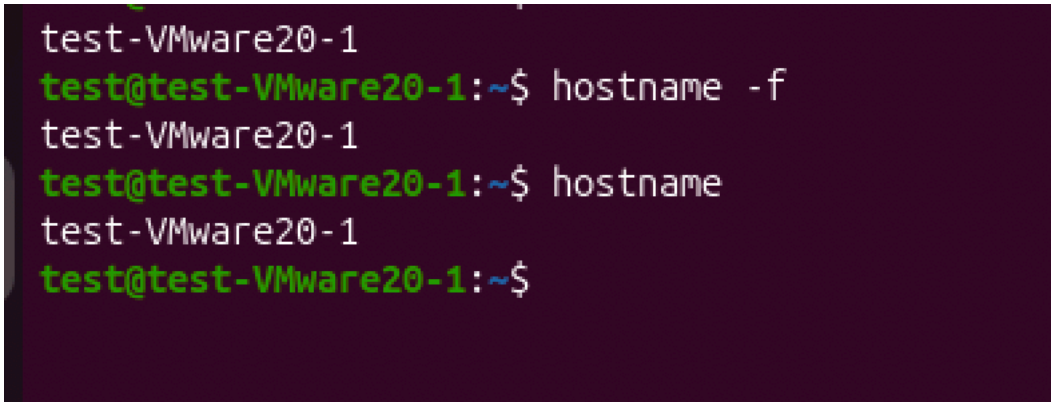
(iii) Afficher le nom de la machine (hostname)

> Le hostname est le "nom" donné à la machine sur le réseau. Comme un prénom pour l'ordinateur.

```
# Nom de la machine
$ hostname

# Sortie attendue :
travel-horizon-srv

# Nom de domaine complet (FQDN)
$ hostname -f
```



2 Afficher les utilisateurs connectés

> Ces commandes permettent de savoir qui est connecté sur le système en ce moment.
> Très utile pour un administrateur qui surveille un serveur.

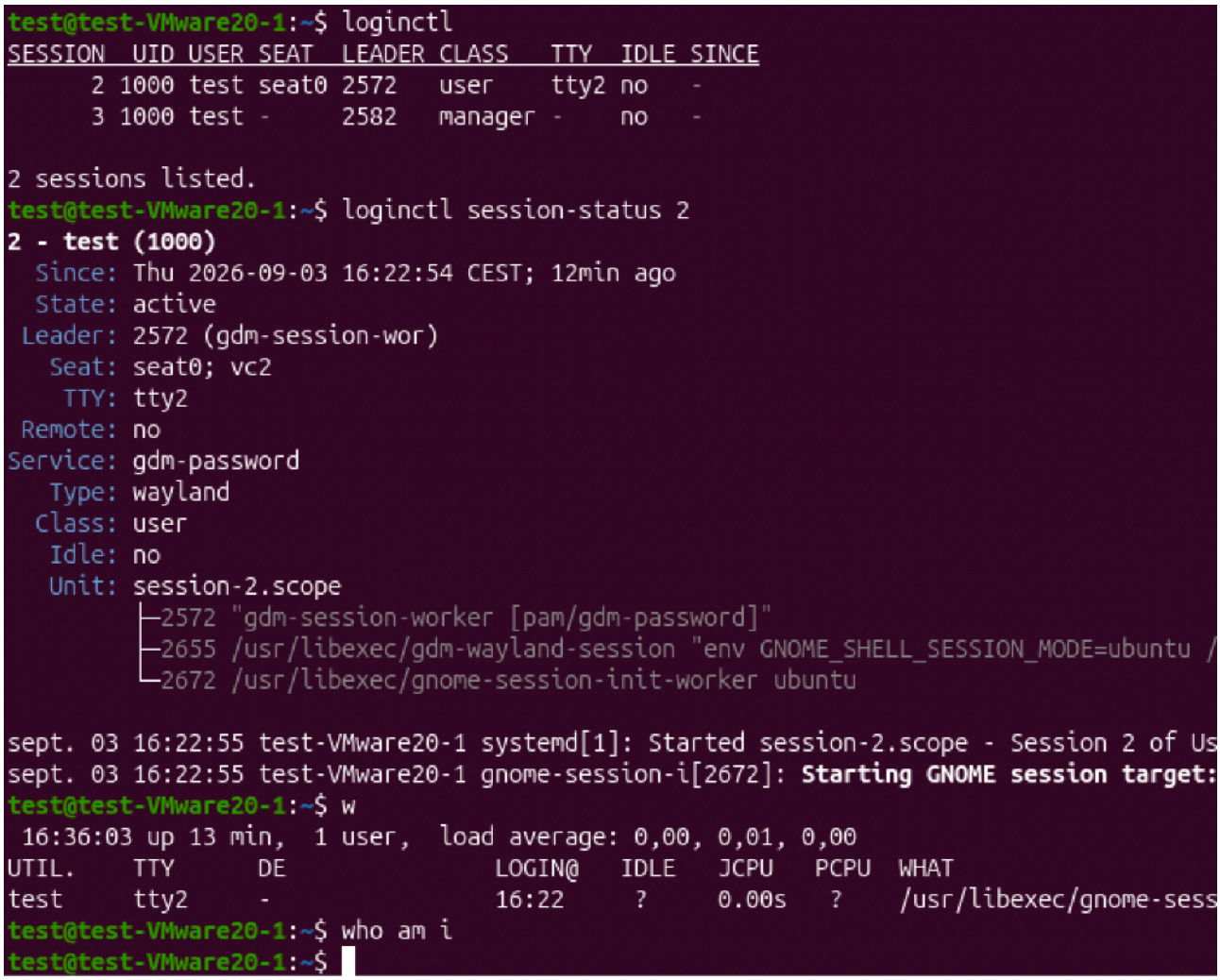
```
# Tous les utilisateurs actuellement connectés
$ who

# Sortie attendue :
Etudinfo pts/0  2026-09-15 09:12 (:0)
alice    pts/1  2026-09-15 09:45 (192.168.1.10)

# Version plus détaillée (heure de connexion, activité...)
$ w

# Chercher un utilisateur particulier (ex: alice)
$ who | grep alice

# Votre propre nom de session
$ who am i
```



Commande	Informations affichées
who	Utilisateurs connectés, terminal, heure de connexion
w	Idem + commande en cours, charge CPU
who | grep nom	Vérifie si un utilisateur précis est connecté
who am i	Votre propre session uniquement
last	Historique des dernières connexions

3 Afficher le calendrier

> La commande cal affiche un calendrier dans le terminal. Utile pour vérifier des dates sans quitter la ligne de commande.

(i) Calendrier de Janvier 2000

```
# cal [mois] [année]
$ cal 1 2000

# Sortie attendue :
    Janvier 2000
Di Lu Ma Me Je Ve Sa
                   1
 2  3  4  5  6  7  8
 9 10 11 12 13 14 15
16 17 18 19 20 21 22
23 24 25 26 27 28 29
30 31
```

(ii) Calendrier de Février 1999

```
$ cal 2 1999
```

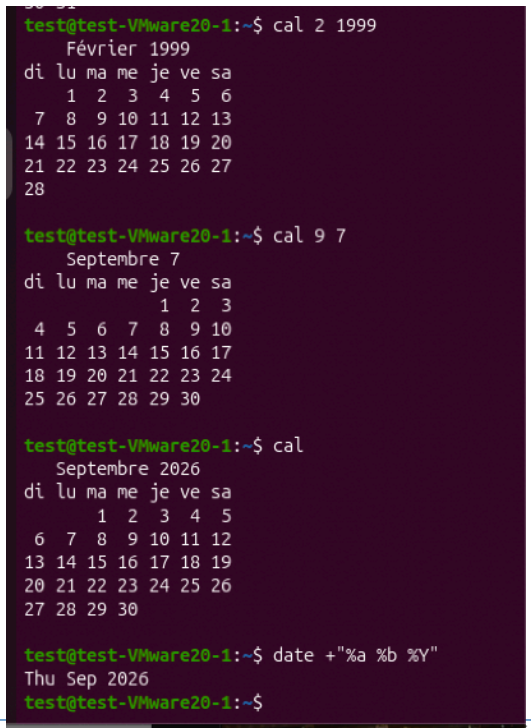
(iii) 9e mois de l'an 7 après J.-C.

```
# Septembre de l'an 7
$ cal 9 7
```

(iv) Mois courant

```
# Sans argument = mois courant
$ cal
```

(v) Date abrégée : jour, mois, année



```
# date avec format personnalisé
# %a = abréviation du jour  %b = abréviation du mois  %Y = année
$ date +"%a %b %Y"

# Sortie attendue (exemple) :
Dim Sep 2026
```

4 Afficher l'heure en format 12h et 24h

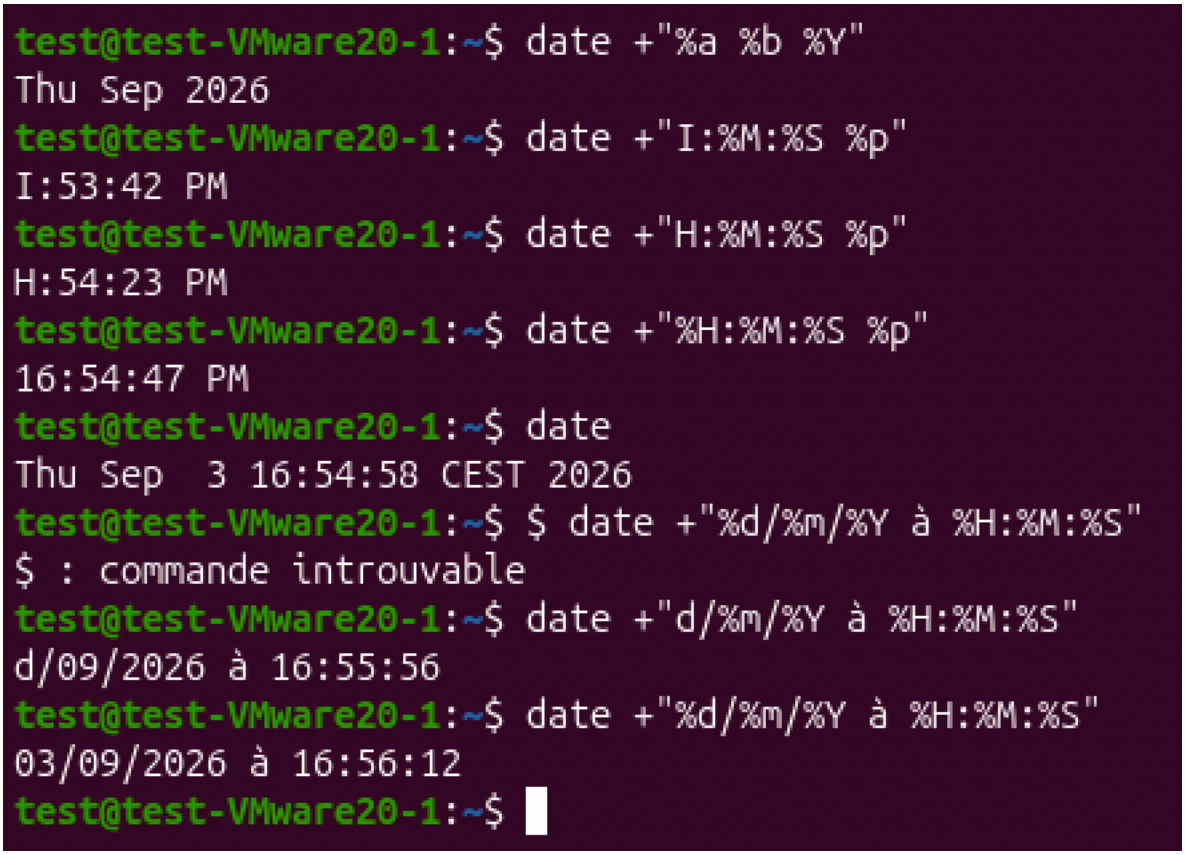
La commande date affiche la date ET l'heure. On peut la formater avec des codes :
%I = heure en format 12h (01-12) %H = heure en format 24h (00-23)
%M = minutes %S = secondes %p = AM ou PM

```
# Format 12 heures (avec AM/PM)
$ date +"%I:%M:%S %p"

# Sortie attendue :
09:35:42 AM

# Format 24 heures
$ date +"%H:%M:%S"

# Sortie attendue :
09:35:42
```



5 Afficher la date et l'heure courantes

```
# Date et heure complètes
$ date

# Sortie attendue :
dim. 15 sept. 2026 09:35:42 CEST

# Format personnalisé complet
$ date +"%d/%m/%Y à %H:%M:%S"

# Sortie attendue :
15/09/2026 à 09:35:42
```

6 Afficher "GOOD MORNING" en grands caractères

> La commande figlet génère du texte en "ASCII art" — des grandes lettres formées par des caractères. Très utilisée pour les messages d'accueil sur les serveurs.

```
Installation de :
  sysvbanner

Sommaire :
  Mise à niveau de : 0. Installation de : 1Supprimé : 0. Non mis à jour : 0
Taille du téléchargement : 7 916 B
  Espace nécessaire : 85,0 kB / 31,1 GB disponible

Réception de : 1 http://ports.ubuntu.com/ubuntu-ports questing/universe arm64 sysvbanner
7 916 o réceptionnés en 0s (34,4 ko/s)
Sélection du paquet sysvbanner précédemment désélectionné.
(Lecture de la base de données... 212456 fichiers et répertoires déjà installés.)
Préparation du dépaquetage de .../sysvbanner_1%3a1.0-18fakesync1_arm64.deb ...
Dépaquetage de sysvbanner (1:1.0-18fakesync1) ...
Paramétrage de sysvbanner (1:1.0-18fakesync1) ...
Traitement des actions différées (« triggers ») pour man-db (2.13.1-1) ...
test@test-VMware20-1:~$ banner "GOOD MORNING"
 #####  ####### ####### ######          #     # ####### ######  #     #   ###
#     # #     # #     # #     #         ##   ## #     # #     # ##    #    #
#       #     # #     # #     #         # # # # #     # #     # # #   #    #
#  #### #     # #     # #     #         #  #  # #     # ######  #  #  #    #
#     # #     # #     # #     #         #     # #     # #   #   #   # #    #
#     # #     # #     # #     #         #     # #     # #    #  #    ##    #
 #####  ####### ####### ######          #     # ####### #     # #     #   ###

test@test-VMware20-1:~$ banner "GOOD MORNING"
 #####  ####### ####### ######          #     # ####### ######  #     #   ###
#     # #     # #     # #     #         ##   ## #     # #     # ##    #    #
#       #     # #     # #     #         # # # # #     # #     # # #   #    #
#  #### #     # #     # #     #         #  #  # #     # ######  #  #  #    #
#     # #     # #     # #     #         #     # #     # #   #   #   # #    #
#     # #     # #     # #     #         #     # #     # #    #  #    ##    #
 #####  ####### ####### ######          #     # ####### #     # #     #   ###
```

```
# Installer figlet si nécessaire
$ sudo apt install figlet -y

# Afficher GOOD MORNING en grands caractères
$ figlet "GOOD MORNING"

# Sortie attendue :
 ____   ___   ___  ____    __  __  ___  ____  _   _  ___  _   _  ____
/ ___) / _ \ / _ \|  _ \  |  \/  |/ _ \|  _ \| \ | ||_ _|| \ | |/ ___)

# Avec une autre police
$ figlet -f slant "GOOD MORNING"

# Alternative avec banner (plus simple)
$ sudo apt install sysvbanner -y
$ banner "GOOD MORNING"
```

Script — Partie A

1. Les administrateurs systèmes aiment personnaliser leur environnement de travail. Un script de bienvenue (ou "motd" - Message Of The Day) affiche automatiquement des informations pertinentes dès l'ouverture du terminal : date, heure, utilisateur, machine, etc.

Votre mission : créer un tel script !

Vous décrivez toutes les étapes de réalisation du script

```
==================================================
            💻  BIENVENUE SUR LINUX  💻
==================================================

📌 INFORMATIONS SYSTÈME
----------------------
  OS            : Linux
  Noyau         : 6.8.0-45-generic
  Architecture  : x86_64
  Version       : #45-Ubuntu

📌 DISTRIBUTION
--------------
  Ubuntu 24.04.1 LTS

📌 UTILISATEUR
-------------
  Login         : etudinfo
  UID/GID       : uid=1000(etudinfo)
  Groupes       : 1000(etudiant),27(sudo)

📌 MACHINE
----------
  Hostname      : travel-horizon-srv
  FQDN          : travel-horizon-srv.local
```

```
📌 DATE ET HEURE
----------------
  Date complète : dim. 15 sept. 2026 09:35:42 CEST
  Format 12h    : 09:35:42 AM
  Format 24h    : 09:35:42
  Calendrier    : Sunday 15 September 2026

📌 CALENDRIER DU MOIS
---------------------
    Septembre 2026
Di Lu Ma Me Je Ve Sa
       1  2  3  4  5
 6  7  8  9 10 11 12
13 14 15 16 17 18 19
20 21 22 23 24 25 26
27 28 29 30

📌 UTILISATEURS CONNECTÉS
-------------------------
  etudinfo pts/0        2026-09-15 09:12 (:0)
  alice    pts/1        2026-09-15 09:45 (192.168.1.10)
```



Partie B — Navigation et gestion des répertoires

Comprendre l'arborescence Linux avant de commencer :
/ → Racine du système (root de l'arbre)
/home/student → Votre répertoire personnel (home directory)
/home/student/SAMPLE → sous-répertoire SAMPLE
/home/student/SAMPLE/TRIAL → sous-sous-répertoire TRIAL
Règle : les chemins absolus commencent par / | les chemins relatifs commencent depuis le dossier courant

7 Afficher le répertoire personnel (home)

```
# echo affiche du texte ou la valeur d'une variable
# $HOME est une variable d'environnement = votre chemin home
$ echo $HOME

# Sortie attendue :
/home/student

# Alternative : pwd depuis le home
$ cd ~
$ pwd

# Sortie attendue :
/home/student
```

Commande / Variable	Signification
echo $HOME	Affiche la valeur de la variable HOME (votre dossier personnel)
~	Raccourci clavier équivalent à $HOME dans les commandes
pwd	Print Working Directory = affiche le chemin du répertoire courant

8 Créer le répertoire SAMPLE dans le home

mkdir = Make Directory. C'est la commande pour créer un nouveau dossier.

```
# Aller dans le home d'abord
$ cd ~

# Créer le répertoire SAMPLE
$ mkdir SAMPLE

# Vérifier qu'il existe
$ ls

# Sortie attendue (SAMPLE visible) :
Desktop  Documents  Downloads  SAMPLE
```

```
test@test-VMware20-1:~$ nano modt.sh
test@test-VMware20-1:~$ echo $HOME
/home/test
test@test-VMware20-1:~$ cd
test@test-VMware20-1:~$ pwd
/home/test
test@test-VMware20-1:~$ cd ~
test@test-VMware20-1:~$ mkdir SAMPLE
test@test-VMware20-1:~$ ls
11-0.txt    Bureau     Images   Musique  Public  Téléc
2701-0.txt  Documents  Modèles  PDF      SAMPLE  Vidéo
test@test-VMware20-1:~$
```

9 Créer le sous-répertoire TRIAL dans SAMPLE

```
# Méthode 1 : entrer dans SAMPLE puis créer TRIAL
$ cd ~/SAMPLE
$ mkdir TRIAL

# Méthode 2 : en une seule commande depuis n'importe où
$ mkdir ~/SAMPLE/TRIAL

# Méthode 3 : créer toute la chaîne d'un coup (-p = parents)
$ mkdir -p ~/SAMPLE/TRIAL

# Vérifier la structure
$ ls ~/SAMPLE

# Sortie attendue :
TRIAL
```

```
test@test-VMware20-1:~$ cd
test@test-VMware20-1:~$ cd /SAMPLE
bash: cd: /SAMPLE: Aucun fichier ou dossier de ce nom
test@test-VMware20-1:~$ cd SAMPLE
test@test-VMware20-1:~/SAMPLE$ mkdir TRIAL
test@test-VMware20-1:~/SAMPLE$ mkdir TRIAL/TRIAL
test@test-VMware20-1:~/SAMPLE$ cd
test@test-VMware20-1:~$ ls SAMPLE/
TRIAL
test@test-VMware20-1:~$
```

> L'option -p de mkdir crée tous les répertoires parents nécessaires. Exemple : mkdir -p /a/b/c/d crée a, puis b, puis c, puis d en une seule commande.

10 Se déplacer dans SAMPLE

cd = Change Directory. C'est la commande pour naviguer dans l'arborescence.

```
# Chemin absolu (commence depuis la racine /)
$ cd /home/student/SAMPLE

# Chemin relatif (depuis le home ~)
$ cd ~/SAMPLE

# Vérifier où on est
$ pwd

# Sortie attendue :
/home/student/SAMPLE
```

11 Revenir au répertoire home

```
# Méthode 1 : cd sans argument → va toujours au home
$ cd

# Méthode 2 : avec le raccourci ~
$ cd ~

# Méthode 3 : avec la variable $HOME
$ cd $HOME

# Vérifier
$ pwd
# Sortie : /home/student
```

12 Aller dans TRIAL depuis le home — chemin absolu ET relatif

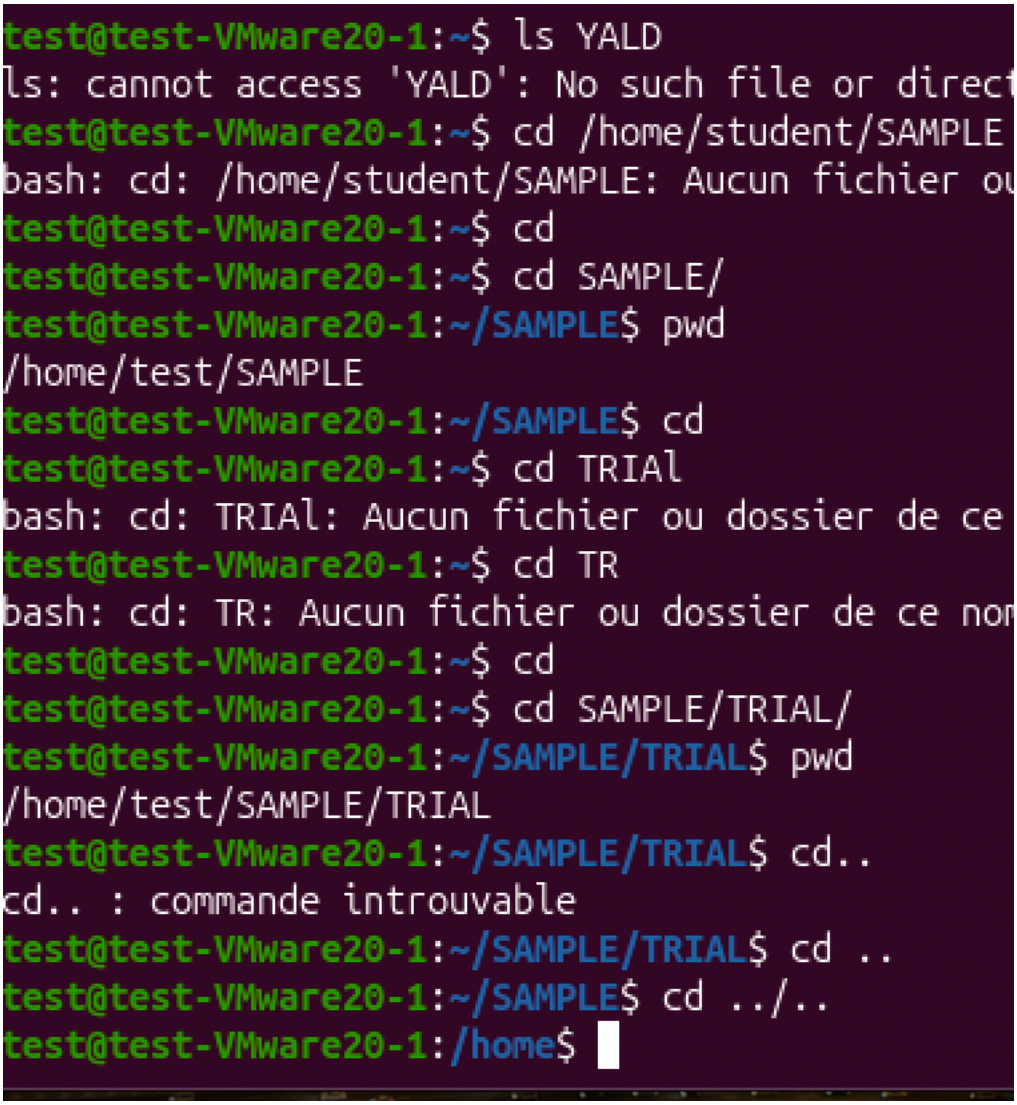
Chemin ABSOLU : part toujours de la racine /, indépendant de votre position actuelle.
Chemin RELATIF : part de votre position actuelle, utilise . (ici) et .. (dossier parent).

```
# ── Chemin absolu ──────────────────────────────────────────
# Commence par / , fonctionne depuis n'importe où
$ cd /home/student/SAMPLE/TRIAL
$ pwd
# Sortie : /home/student/SAMPLE/TRIAL

# ── Chemin relatif ─────────────────────────────────────────
# D'abord revenir au home
$ cd ~

# Puis naviguer relativement
$ cd SAMPLE/TRIAL
$ pwd
# Sortie : /home/student/SAMPLE/TRIAL

# ── Remonter avec .. ───────────────────────────────────────
# .. = répertoire parent
$ cd ..         # remonte dans SAMPLE
$ cd ../..      # remonte deux niveaux (dans /home/student)
```



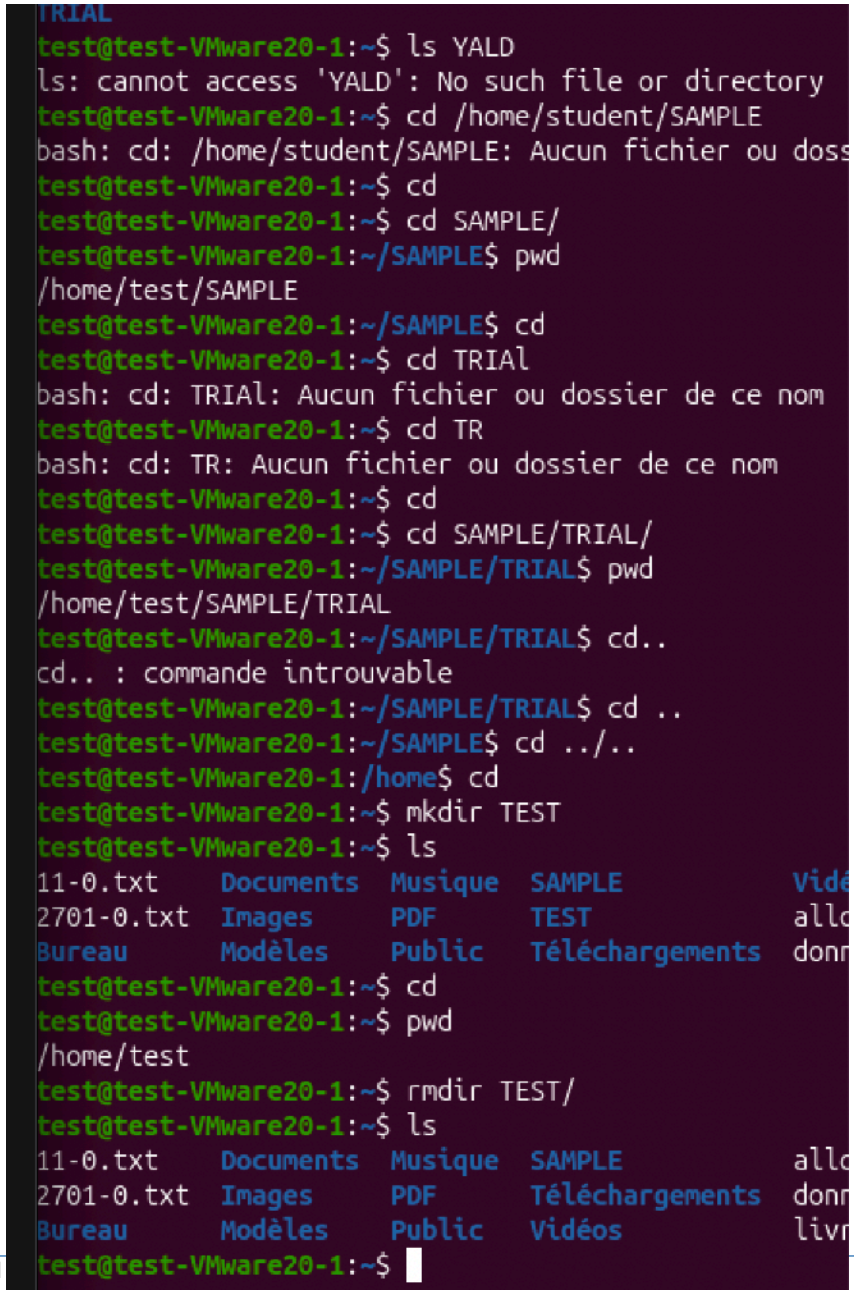
13 Créer le répertoire TEST par chemin absolu

```
# Depuis n'importe quel répertoire courant,
# un chemin absolu crée le dossier exactement où indiqué
$ mkdir /home/student/TEST

# Ou avec ~
$ mkdir ~/TEST

# Vérifier
$ ls ~
# Sortie attendue :
SAMPLE  TEST  Desktop  Documents ...
```

14 Revenir au home en une seule commande



```
# cd sans argument = raccourci universel vers $HOME
$ cd

# Vérifier
$ pwd
# Sortie : /home/student
```

15 Supprimer un répertoire par chemin absolu

rmdir = Remove Directory. Ne fonctionne que sur un répertoire VIDE.
Pour supprimer un répertoire avec des fichiers : rm -rf (voir Tâche 31).

```
# Supprimer le répertoire TEST (vide)
$ rmdir /home/student/TEST

# Vérifier
$ ls ~
# TEST ne doit plus apparaître

# Si TEST contient des fichiers, rmdir échouera :
# rmdir: failed to remove '/home/student/TEST': Directory not empty
```

Questions de compréhension — Partie B

1. Quelle différence entre cd ~ et cd .. ?

cd ~ renvoie directement à la **racine de votre dossier personnel** (votre répertoire *home*, équivalent à /home/votre_login ou $HOME), quel que soit l'endroit où vous vous trouvez dans le système de fichiers.

- **cd ..** remonte d'**un seul niveau dans l'arborescence**, c'est-à-dire vers le répertoire parent direct de votre dossier actuel.

2. Depuis /home/student/SAMPLE/TRIAL, quel chemin relatif permet d'aller dans /home/student ?

- Le chemin relatif est : **../..**
- *Explication :* Le premier .. remonte dans SAMPLE, et le second .. remonte dans /home/student. En commande complète : cd ../..

3. Pourquoi rmdir échoue-t-il si le répertoire contient des fichiers ?

Par conception et pour des raisons de **sécurité anti-erreur humaine**, rmdir (*Remove Directory*) a pour règle stricte de ne supprimer que des répertoires **totalement vides**. Cela évite d'effacer accidentellement des arborescences de données importantes. Pour supprimer un dossier non vide, il faut utiliser la commande récursive rm -r (ou rm -rf).

4. Que fait la commande mkdir -p a/b/c/d si le dossier b existe déjà ?

L'option **-p** (*parents*) fait deux choses :

1. Elle **ne renvoie aucune erreur** si un ou plusieurs dossiers du chemin existent déjà (comme a ou b).
2. Elle crée uniquement les dossiers qui manquent dans la chaîne (ici, elle créera c, puis d à l'intérieur de b).

Partie C — Création et gestion des fichiers

16 Créer les fichiers myfile et yourfile

Deux façons courantes de créer un fichier :
- touch : crée un fichier vide (ou met à jour la date si existe déjà)
- cat > fichier : crée un fichier et attend que vous tapiez du contenu (Ctrl+D pour finir)

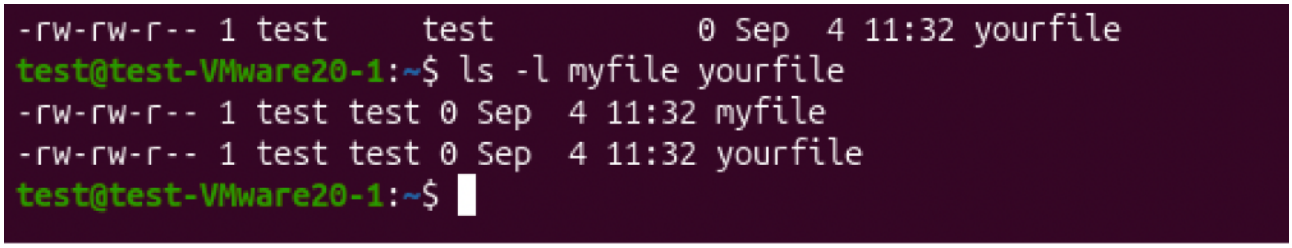
```
# S'assurer d'être dans le bon répertoire
$ cd ~

# Méthode 1 : créer des fichiers vides avec touch
$ touch myfile yourfile

# Méthode 2 : créer avec contenu avec cat >
$ cat > myfile
Bonjour, ceci est myfile.
Ligne 2 de myfile.
# Appuyer sur Ctrl+D pour sauvegarder et quitter

$ cat > yourfile
Bonjour, ceci est yourfile.
Ligne 2 de yourfile.
# Ctrl+D

# Vérifier
$ ls -l myfile yourfile
```



17 Afficher le contenu des fichiers

cat = Concatenate. Affiche le contenu d'un fichier dans le terminal. C'est l'un des outils les plus utilisés en administration Linux.

```
# Afficher myfile
$ cat myfile

# Afficher yourfile
$ cat yourfile

# Afficher les deux à la suite
$ cat myfile yourfile

# Afficher avec numéros de ligne
$ cat -n myfile

# Afficher par pages (si fichier long)
$ less myfile
# Naviguer : espace = page suivante, q = quitter
```

18 Ajouter des lignes dans myfile et yourfile

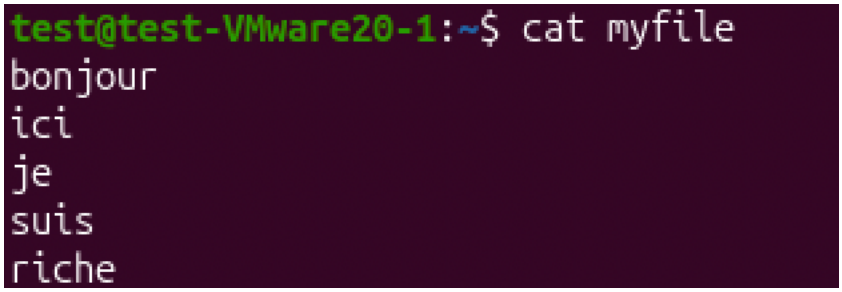
Attention à la différence :
> (redirection simple) : ÉCRASE le fichier existant
>> (redirection double) : AJOUTE à la fin (append) sans effacer

```
# Ajouter des lignes à myfile (>> = append)
$ cat >> myfile
Troisième ligne ajoutée.
Quatrième ligne ajoutée.
# Ctrl+D

# Ajouter à yourfile
$ cat >> yourfile
Nouvelle ligne dans yourfile.
# Ctrl+D

# Vérifier le contenu après ajout
$ cat myfile

# Alternative avec echo >>
$ echo "Une ligne supplémentaire" >> myfile
```



19 Créer un fichier caché

Sous Linux, tout fichier dont le nom commence par un point (.) est automatiquement caché. Il n'apparaît pas avec ls ordinaire, mais avec ls -a.

```
# Créer un fichier caché (préfixe .)
$ touch .fichier_cache

# ls ordinaire : ne le voit pas
$ ls

# ls -a (all) : voit les fichiers cachés
$ ls -a

# Sortie attendue :
.  ..  .fichier_cache  myfile  yourfile

# ls -la : caché + détails
$ ls -la
```

Les fichiers de configuration Linux sont souvent cachés : .bashrc, .profile, .ssh/, .gitconfig... C'est une convention Unix, pas une vraie protection de sécurité.

20 Copier myfile vers emp

cp = Copy. Copie un fichier ou répertoire. La source reste intacte, une nouvelle copie est créée.

```
# cp source destination
$ cp myfile emp

# Vérifier
$ ls
# myfile et emp existent tous les deux

# Afficher emp pour confirmer le contenu copié
$ cat emp

# cp avec confirmation si écrasement (-i = interactive)
$ cp -i myfile emp
```

21 Créer un alias (lien symbolique) pour un fichier

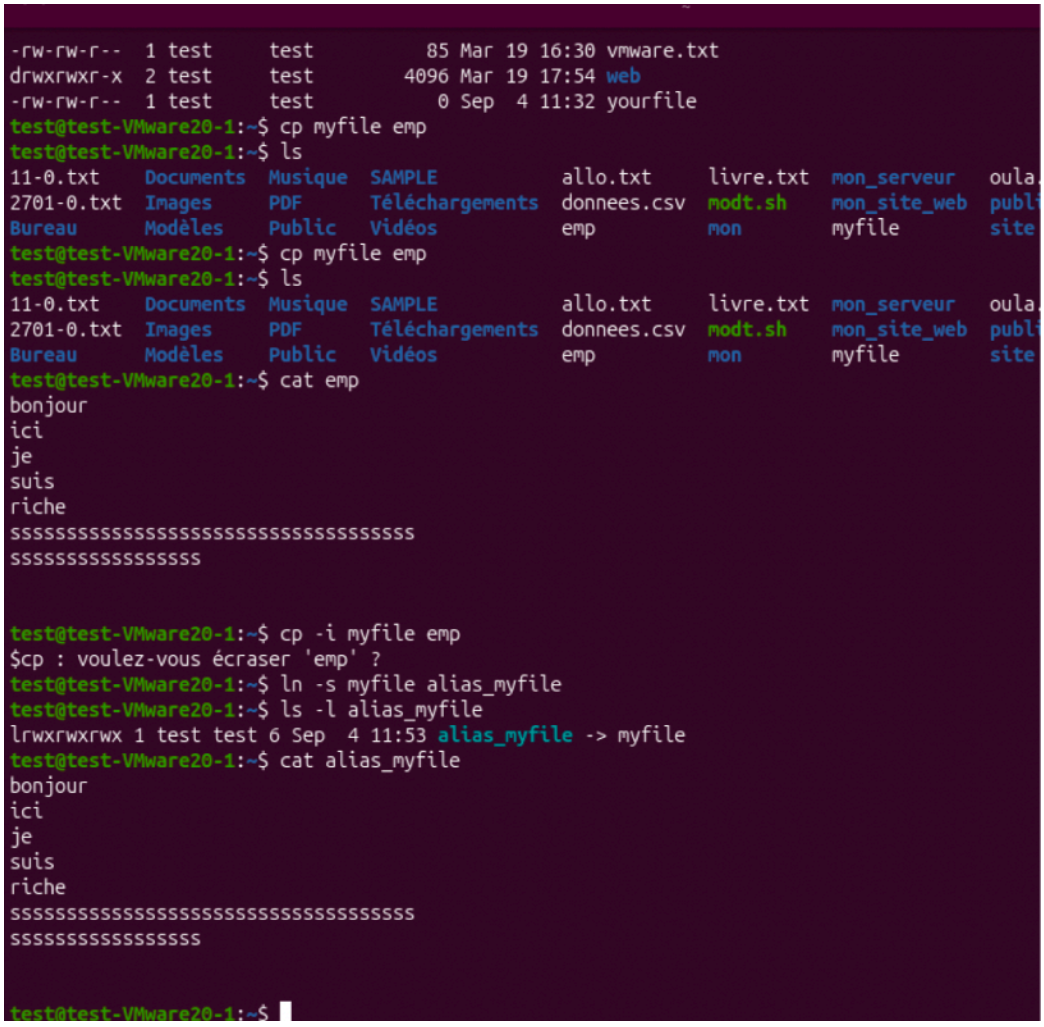
Un lien symbolique (alias) est un raccourci qui pointe vers un autre fichier.
ln -s source lien → crée un alias "lien" qui pointe vers "source"
Modifier l'alias modifie l'original. Supprimer l'alias ne supprime pas l'original.

```
# ln -s = symbolic link (lien symbolique = alias)
# ln -s fichier_original nom_du_lien
$ ln -s myfile alias_myfile

# Vérifier : ls -l montre la flèche ->
$ ls -l alias_myfile

# Sortie attendue :
lrwxrwxrwx 1 student student 6 sept. 15 09:00 alias_myfile -> myfile

# Le contenu est identique
$ cat alias_myfile
```



Type de lien	Commande	Caractéristique principale
Lien symbolique (souple)	ln -s source lien	Raccourci (comme un alias) — peut pointer vers répertoires
Lien physique (dur)	ln source lien	Même inode que l'original — survit à la suppression de l'original

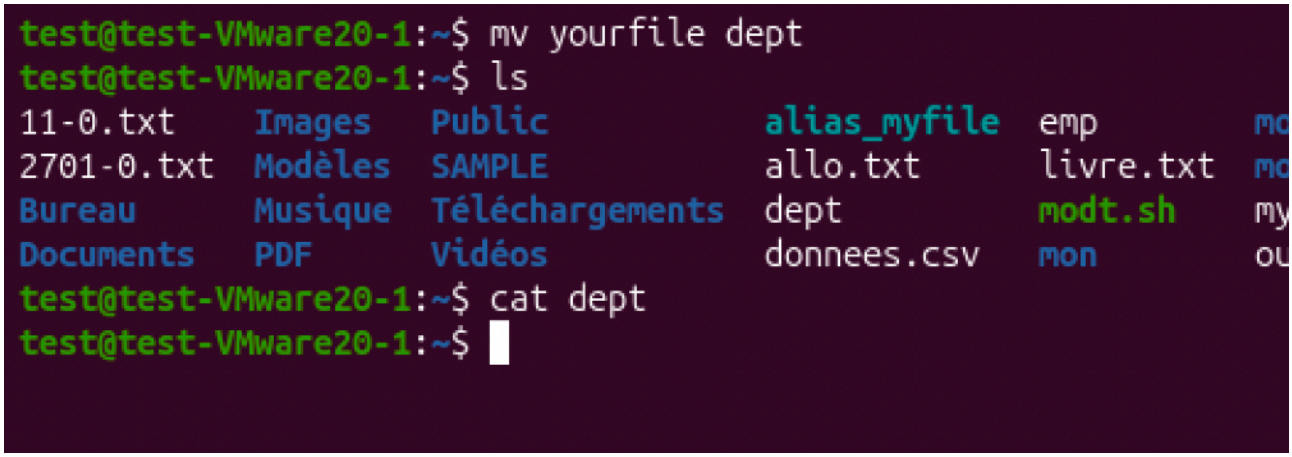
22 Déplacer yourfile vers dept

mv = Move. Déplace OU renomme un fichier. Contrairement à cp, la source disparaît.

```
# mv source destination
$ mv yourfile dept

# Vérifier : yourfile n'existe plus
$ ls
# Sortie attendue : dept  emp  myfile (plus yourfile)

# Afficher le contenu de dept
$ cat dept
# Contenu identique à yourfile
```



mv sert aussi à renommer : mv ancien_nom nouveau_nom dans le même répertoire = renommage.

23 Copier emp et dept dans le répertoire TRIAL

```
# Copier deux fichiers vers un répertoire
$ cp emp dept ~/SAMPLE/TRIAL/

# Vérifier dans TRIAL
$ ls ~/SAMPLE/TRIAL/

# Sortie attendue :
dept  emp
```

24 Comparer des fichiers avec diff

diff compare deux fichiers ligne par ligne et affiche les différences. Si les fichiers sont identiques, aucune sortie n'est produite.



```
# ── Tâche 25 : comparer un fichier avec lui-même ──────────────
$ diff myfile myfile
# Aucune sortie = fichiers identiques

# ── Tâche 26 : comparer myfile et emp ─────────────────────────
$ diff myfile emp
# Aucune sortie attendue (emp est une copie de myfile)

# Lecture de la sortie diff (si différences) :
# < = ligne dans le 1er fichier
# > = ligne dans le 2e fichier
# --- = séparateur

# Exemple de sortie avec différences :
# 3c3
# < Troisième ligne de myfile
# ---
# > Troisième ligne de emp (différente)
```

25 Ajouter deux lignes dans emp dans TRIAL

```
# Ajouter des lignes à emp qui est DANS TRIAL
$ cat >> ~/SAMPLE/TRIAL/emp
Ligne ajoutée directement dans TRIAL/emp.
Deuxième ligne ajoutée.
# Ctrl+D

# Vérifier
$ cat ~/SAMPLE/TRIAL/emp
```

26 Comparer les fichiers emp du home et de TRIAL

```
# Tâche 28 : comparer emp du home avec emp dans TRIAL
$ diff ~/emp ~/SAMPLE/TRIAL/emp

# Sortie attendue (les 2 lignes ajoutées apparaissent) :
3a4,5
> Ligne ajoutée directement dans TRIAL/emp.
> Deuxième ligne ajoutée.

# Tâche 29 : diff avec sortie formatée pour meilleure lisibilité
$ diff -u ~/emp ~/SAMPLE/TRIAL/emp

# -u = format unifié (plus lisible) :
# --- emp (version home)
# +++ TRIAL/emp (version TRIAL)
# -ligne supprimée  +ligne ajoutée
```

```
test@test-VMware20-1:~$ cat SAMPLE/TRIAL/emp
bonjour
ici
je
suis
riche
sssssssssssssssssssssssssssssssss
sssssssssssssss


allo
allo
dddddd
test@test-VMware20-1:~$ diff emp SAMPLE/TRIAL/emp
9a10,12
> allo
> allo
> dddddd
test@test-VMware20-1:~$ diff -u emp SAMPLE/TRIAL/emp
--- emp 2026-09-04 11:52:40.957163546 +0200
+++ SAMPLE/TRIAL/emp    2026-09-07 10:50:48.732516372 +0200
@@ -7,3 +7,6 @@
 sssssssssssssss


+allo
+allo
+dddddd
```

Questions de compréhension — Partie C

1. Quelle différence entre > et >> dans une redirection ?

1. Différence entre > et >>

- **>** **écrase** le fichier cible avec le nouveau contenu (s'il existe déjà, son ancien texte est perdu).
- **>>** **ajoute** à la fin du fichier cible sans rien supprimer de l'existant.

2. Que se passe-t-il si on utilise cp sans l'option -i sur un fichier existant ?

`cp` sans l'option `-i` sur un fichier existant Il **écrase directement** le fichier cible sans demander de confirmation. L'option `-i` (*interactive*) sert précisément à te demander : « Voulez-vous écraser ? ».

3. mv yourfile dept : yourfile existe-t-il encore après cette commande ?

Non, il n'existe plus sous ce nom. `mv` déplace ou renomme. Soit `yourfile` est devenu `dept` (renommage), soit il a été déplacé dans le dossier `dept` si ce dernier existait déjà.

4. Comment créer un fichier caché nommé .secret ?

`.secret` Sous Linux, un point **`.`** au début du nom suffit pour le rendre caché.

Touch .secret

Partie D — Suppression, renommage et liens

27 Supprimer les fichiers dans TRIAL

rm = Remove. Supprime des fichiers. ATTENTION : sous Linux, la suppression est définitive, il n'y a pas de corbeille en ligne de commande !

```
# Supprimer un fichier spécifique dans TRIAL
$ rm ~/SAMPLE/TRIAL/emp

# Supprimer plusieurs fichiers
$ rm ~/SAMPLE/TRIAL/emp ~/SAMPLE/TRIAL/dept

# Supprimer avec confirmation (-i = interactive)
$ rm -i ~/SAMPLE/TRIAL/emp
# Le système demande : remove 'emp'? → taper y puis Entrée

# Vérifier
$ ls ~/SAMPLE/TRIAL/
# Répertoire vide attendu
```

28 Supprimer un répertoire avec des fichiers en une seule commande

rm -rf = Remove Recursive Force
-r = récursif (supprime le contenu et les sous-dossiers)
-f = force (pas de confirmation, pas d'erreur si absent)
DANGER : rm -rf /* supprimerait tout le système ! Toujours vérifier avant d'exécuter.

```
# rmdir échoue si le répertoire n'est pas vide :
# rmdir: failed to remove 'TRIAL': Directory not empty

# rm -r supprime un répertoire et tout son contenu
$ rm -r ~/SAMPLE/TRIAL

# rm -rf : sans demande de confirmation
$ rm -rf ~/SAMPLE/TRIAL

# Bonne pratique : toujours lister avant de supprimer
$ ls ~/SAMPLE/TRIAL   # vérifier ce qu'on va supprimer
$ rm -rf ~/SAMPLE/TRIAL
```

29 Peut-on récupérer un fichier supprimé ?

Réponse courte : NON, pas facilement. Sous Linux en ligne de commande :
- rm supprime définitivement (pas de corbeille)
- Récupération possible via des outils forensiques (testdisk, extundelete) mais complexe
- La règle d'or : faire des SAUVEGARDES régulières (tar, rsync, Timeshift)

```
# Il n'existe pas de commande "undelete" native sous Linux

# Outil de récupération forensique (si filesystem ext4)
$ sudo apt install extundelete -y
$ sudo extundelete /dev/sda1 --restore-file home/student/emp

# Outil plus général
$ sudo apt install testdisk -y
$ sudo testdisk

# PRÉVENTION : créer une sauvegarde AVANT de supprimer
$ cp emp emp.backup
$ tar czf sauvegarde.tar.gz ~/SAMPLE/
```

30 Renommer TRIAL en DATA

Pour renommer un répertoire sous Linux, on utilise mv (même commande que pour déplacer). mv ancien_nom nouveau_nom dans le même dossier = renommage.

```
# Recréer TRIAL pour cet exercice
$ mkdir ~/SAMPLE/TRIAL

# Renommer TRIAL en DATA
$ mv ~/SAMPLE/TRIAL ~/SAMPLE/DATA

# Vérifier
$ ls ~/SAMPLE/
# Sortie attendue :
DATA

# TRIAL n'existe plus, DATA existe
```

31 Copier DATA vers un nouveau répertoire TRIAL

Pour copier un répertoire entier avec cp, l'option -r (récursif) est obligatoire. Sans -r, cp ne copie pas les dossiers.

```
# cp -r = copie récursive (répertoire + contenu)
$ cp -r ~/SAMPLE/DATA ~/SAMPLE/TRIAL

# Vérifier
$ ls ~/SAMPLE/
# Sortie attendue :
DATA  TRIAL

# DATA et TRIAL existent tous les deux
```

32 Liens physiques et symboliques

Rappel des deux types de liens :
ln -s source lien → Lien SYMBOLIQUE (souple) : comme un raccourci Windows
ln source lien → Lien PHYSIQUE (dur) : deux noms pour le même fichier

Créer dummy dans TRIAL et le lier à star

```
# Créer le fichier dummy dans TRIAL
$ cat > ~/SAMPLE/TRIAL/dummy
Contenu du fichier dummy.
# Ctrl+D

# Créer un lien symbolique star qui pointe vers dummy
$ ln -s ~/SAMPLE/TRIAL/dummy ~/SAMPLE/TRIAL/star

# Vérifier
$ ls -l ~/SAMPLE/TRIAL/

# Sortie attendue :
-rw-r--r-- 1 student student 26 sept. 15 dummy
lrwxrwxrwx 1 student student 30 sept. 15 star -> /home/student/SAMPLE/TRIAL/
dummy

# Lire via le lien = lire l'original
$ cat ~/SAMPLE/TRIAL/star
```

Lier dummy (TRIAL) à power dans DATA

```
test@test-VMware20-1:~$ rm -r SAMPLE/TRIAL/
test@test-VMware20-1:~$ rm -rf /SAMPLE/TRIAL
test@test-VMware20-1:~$ ls /SAMPLE/TRIAL
ls: cannot access '/SAMPLE/TRIAL': No such file or directory
test@test-VMware20-1:~$ ls ~/SAMPLE
test@test-VMware20-1:~$ mkdir ~/SAMPLE/TRIAL
mv ~/SAMPLE/TRIAL ~/SAMPLE/DATA
ls ~/SAMPLE/
DATA
test@test-VMware20-1:~$ cp -r ~/SAMPLE/DATA ~/SAMPLE/TRIAL
ls ~/SAMPLE/
DATA  TRIAL
test@test-VMware20-1:~$ echo "Contenu du fichier dummy." > ~/SAMPLE/TRIAL/dummy
test@test-VMware20-1:~$ ln -s ~/SAMPLE/TRIAL/dummy ~/SAMPLE/TRIAL/star
test@test-VMware20-1:~$ ls -l ~/SAMPLE/TRIAL/
total 4
-rw-rw-r-- 1 test test 26 Sep  7 11:14 dummy
lrwxrwxrwx 1 test test 29 Sep  7 11:14 star -> /home/test/SAMPLE/TRIAL/dummy
test@test-VMware20-1:~$ cat ~/SAMPLE/TRIAL/star
Contenu du fichier dummy.
test@test-VMware20-1:~$ ln ~/SAMPLE/TRIAL/dummy ~/SAMPLE/DATA/power
test@test-VMware20-1:~$ ls -li ~/SAMPLE/TRIAL/dummy ~/SAMPLE/DATA/power
660147 -rw-rw-r-- 2 test test 26 Sep  7 11:14 /home/test/SAMPLE/DATA/power
660147 -rw-rw-r-- 2 test test 26 Sep  7 11:14 /home/test/SAMPLE/TRIAL/dummy
test@test-VMware20-1:~$ echo "Ajout via power" >> ~/SAMPLE/DATA/power
cat ~/SAMPLE/TRIAL/dummy
Contenu du fichier dummy.
Ajout via power
test@test-VMware20-1:~$
```

```
# Lien PHYSIQUE de dummy (TRIAL) vers power (DATA)
$ ln ~/SAMPLE/TRIAL/dummy ~/SAMPLE/DATA/power

# Vérifier : ls -li montre le numéro d'inode
$ ls -li ~/SAMPLE/TRIAL/dummy ~/SAMPLE/DATA/power

# Sortie attendue (même inode = même fichier) :
123456 -rw-r--r-- 2 student student 26 ... /home/student/SAMPLE/DATA/power
123456 -rw-r--r-- 2 student student 26 ... /home/student/SAMPLE/TRIAL/dummy
# Le chiffre 2 = nombre de liens vers cet inode

# Modifier power modifie dummy et vice-versa
$ echo "Ajout via power" >> ~/SAMPLE/DATA/power
$ cat ~/SAMPLE/TRIAL/dummy
# La ligne ajoutée apparaît !
```

Critère	Lien symbolique (ln -s)	Lien physique (ln)
Commande	ln -s source lien	ln source lien
Fonctionne sur répertoires	✅ Oui	❌ Non (en général)
Si original supprimé	❌ Lien cassé (dangling)	✅ Fichier toujours accessible
Filesystems différents	✅ Oui	❌ Non (même FS obligatoire)
Visible dans ls -l	-> (flèche)	Même taille/inode

Synthèse — Tableau de bord des commandes

Commande	Syntaxe principale	Rôle	Partie
uname	uname -a	Informations sur le noyau et l'OS	A
whoami	whoami	Nom de l'utilisateur courant	A
hostname	hostname	Nom de la machine	A
who / w	who | w	Utilisateurs connectés	A
cal	cal [mois] [année]	Afficher un calendrier	A
date	date +"%format"	Date et heure (formatées)	A
figlet	figlet "texte"	Texte en gros caractères ASCII	A
echo	echo $VARIABLE	Afficher texte ou variable	B
pwd	pwd	Répertoire courant	B
cd	cd [chemin]	Changer de répertoire	B
mkdir	mkdir [-p] répertoire	Créer un répertoire	B
rmdir	rmdir répertoire	Supprimer répertoire vide	B
ls	ls [-la]	Lister les fichiers	B/C
touch	touch fichier	Créer fichier vide	C
cat	cat [>|>>] fichier	Créer / lire / ajouter dans fichier	C
cp	cp [-r] source dest	Copier fichier ou répertoire	C
mv	mv source dest	Déplacer ou renommer	C
diff	diff [-u] f1 f2	Comparer deux fichiers	C
rm	rm [-rf] fichier	Supprimer fichier ou répertoire	D
ln	ln source lien	Lien physique	D
ln -s	ln -s source lien	Lien symbolique (alias)	D

Aide-mémoire rapide — Navigation

```
# Raccourcis essentiels
$ cd           # → home
$ cd ~         # → home
$ cd ..        # → répertoire parent
$ cd -         # → répertoire précédent
$ pwd          # → où suis-je ?
$ ls -la       # → que contient ce dossier ? (cachés inclus)

# Chemins
# Absolu  : /home/student/SAMPLE/TRIAL  (commence par /)
# Relatif : SAMPLE/TRIAL                (depuis position actuelle)
# Tilde   : ~/SAMPLE/TRIAL              (depuis le home)
```

Récapitulatif des redirections

Symbole	Nom	Effet	Exemple
>	Redirection sortie	Crée ou ÉCRASE le fichier	echo "hello" > fichier
>>	Redirection append	AJOUTE à la fin du fichier	echo "hello" >> fichier
<	Redirection entrée	Lit depuis un fichier	cat < fichier
|	Pipe	Envoie la sortie vers une autre commande	who | grep alice

Compétences BTS SIO SISR mobilisées

Bloc de compétences	Activités couvertes
B2 — Administration des serveurs Linux	Navigation, gestion fichiers/répertoires, shell
B2 — Utilisation des commandes système	Informations système, date, utilisateurs, liens
B2 — Organisation du système de fichiers	Chemins absolus/relatifs, arborescence, liens
B3 — Documentation technique	Compte-rendu des commandes, tableaux de vérification

Mission d'auto-évaluation

Sans regarder ce document, répondez aux questions suivantes en tapant les commandes :

1. Quelle commande affiche le répertoire courant ?

```
pwd
```

2. Créer la structure : ~/PROJET/SRC/CODE en une seule commande

```
mkdir -p ~/PROJET/SRC/CODE
```

3. Créer un fichier readme.txt avec le contenu "Mon projet »

```
echo "Mon projet" > readme.txt
```

4. Copier readme.txt dans ~/PROJET/SRC/

```
cp readme.txt ~/PROJET/SRC/
```

5. Afficher les différences entre les deux copies

```
diff readme.txt ~/PROJET/SRC/readme.txt
```

6. Créer un lien symbolique raccourci vers ~/PROJET/SRC/readme.txt

```
ln -s ~/PROJET/SRC/readme.txt raccourci
```

7. Supprimer ~/PROJET et tout son contenu en une seule commande

```
rm -rf ~/PROJET
```